中山大學
SUN YAT-SEN UNIVERSITY



继承和派生

中山大学计算机学院

主讲人：姓名

中山大学MOOC课程组

目录

CONTENTS

01 继承/派生的概念

02 语法与访问控制

03 构造与析构顺序

04 派生与构造函数

05 派生与成员函数

06 改变访问控制

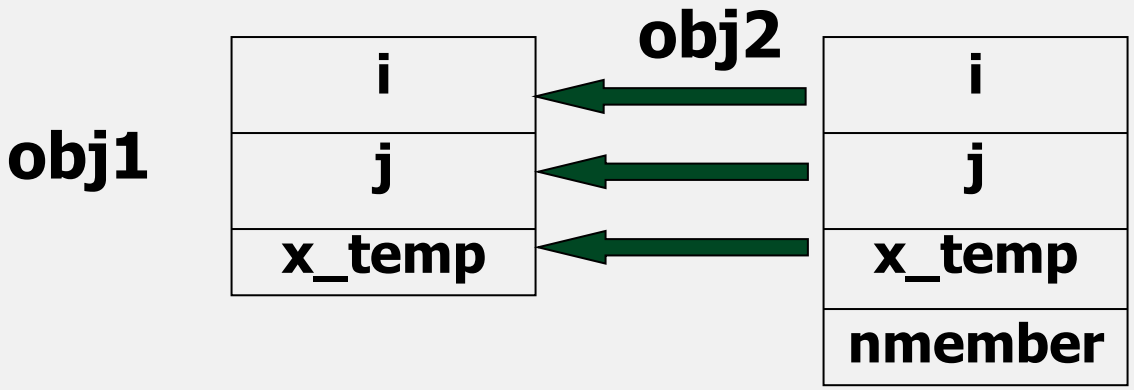
07 类型兼容性

08 对象的类型转换

09 多重继承

类型兼容性（向上兼容）

- 赋值运算的类型兼容性
  - 可以将后代类的对象赋值给祖先类对象，反之不可。
  - 每个派生类对象包含一个基类部分，这意味着可以将派生类对象当作基类对象使用。




```
BASE   obj1;
Y1     obj2;
obj1 = obj2;   //把obj2中基类部分的内容赋给obj1
obj2 = obj1 ;  //错误
```

类型兼容性

```
Y1继承BASE，且
    BASE  obj1;
    Y1    obj2;
```

- 基类类型的**指针或引用** 可指向/引用公有派生类型的对象

```
BASE *p ;
p = &obj1;  ✓
p = &obj2;  ✓
p = p1 ;    ✓

Y1 *p1;
p1 = &obj1;  ✗
p1 = &obj2;  ✓
```

- 只有公有派生类才能兼容基类类型（上述规则只适用于公有派生）。

类型兼容性例子

```
class Base {     //B.h
public:
        void display() ;
};

class D1: public Base {
 public:
        void display() ;
};

class D2: public D1 {
public:
        void display() ;
};
```

```
//B.cpp

#include “B.h”

void Base::display()
{    cout<<"Base::display()"<<endl;  }

void D1:: display()
{    cout<<"B2::display()"<<endl;      }

void D2:: display()
{    cout<<"D2::display()"<<endl;      }
```

类型兼容性例子

```
void fun(Base *ptr)
{      ptr->display();     }

int main()
{      Base b;          //声明B0类对象
       D1 d1;           //声明D1类对象
       D2 d2;           //声明D2类对象
       Base *p;         //声明Base类指针
       p=&b;            //Base类指针指向Base类对象
       fun(p);
       p=&d1;           //Base类指针指向D1类对象
       fun(p);
       p=&d2;           //Base类指针指向D2类对象
       fun(p);
}
```

运行结果:
Base::display()
Base::display()
Base::display()

解释:
形参是指针类型，其基类型为Base

目录

CONTENTS

01 继承/派生的概念

02 语法与访问控制

03 构造与析构顺序

04 派生与构造函数

05 派生与成员函数

06 改变访问控制

07 类型兼容性

08 对象的类型转换

09 多重继承

Upcasting and Downcasting

- Upcasting（向 上/基 类型转换）
  - Assigning a pointer of a derived class type to a pointer of its base class type.Done implicitly

```
GeometricObject *p = new Circle(1);
Circle *p1 = new Circle(2);
p = p1;
```

指针：基类型 派生类型

- Downcasting（向 下/派生 类型转换）
  - Assigning a pointer of a base class type to a pointer of its derived class type
  - Done explicitly using dynamic_cast

```
p1 = dynamic_cast<Circle *>(p);
```

指针：派生类 基类

Dynamic Casting

- **The display function：**

```
// A function for displaying a geometric object
void display(Figure *p) {
  cout << "The area is " << p->get_area() << endl;
}
```

```
// A function for displaying a geometric object
void display(Figure *p) {
  Circle* circlePtr =  p;//?
  cout << "The area is " << circlePtr ->get_area() <<
endl;
}
```

Casts

- static_cast
  - Used to convert one data type to another and hands all reasonable casts

```
average = (float) hits / (float)
at_bats;
average = static_cast<float>(hits) /
static_cast<float>(at_bats);
```

Casts

- const_cast
  - Used to cast away constness.

```
#include <iostream>
using namespace std;
int main() {
   const int i = 100;
   const int *p = &i;
   int *q = const_cast<int*>(p);
   int j = i;

   cout << i << endl << j<<endl
        << *p << endl  << *q <<endl;
  return 0;
}
```

```
#include <iostream>
using namespace std;
int main() {
   const int i = 100;
   const int &p = i;
   int &q = const_cast<int&>(p);
   int j = i;

   cout << i << endl << j << endl
        << p << endl << q <<endl;
   return 0;
}
```

Casts

- reinterpret_cast

converts between unrelated types such as an integer to a pointer or a pointer to an unrelated pointer type.

void* and reinterpret_cast

```
int *ip;
char *cp;
void *gp;
cp=ip;
ip=cp;
cp=gp;
gp=cp;
cp=reinterpret_cast<char*>(gp);
```

注意 C 与 C++区别:

- C语言 void * 可以隐式转型任何类型指针
- C++的 void * 需要显式转型其他类型指针

Casts

- **dynamic_cast**
  - Used for casting across or within inheritance.
  - This cast is used with classes having virtual functions.

目录

CONTENTS

01 继承/派生的概念

02 语法与访问控制

03 构造与析构顺序

04 派生与构造函数

05 派生与成员函数

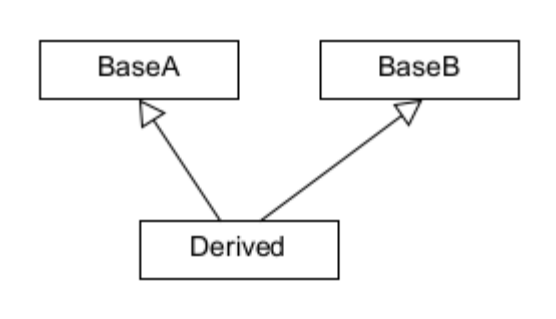
06 改变访问控制

07 类型兼容性

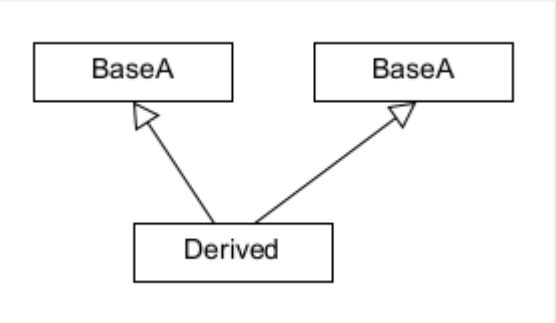
08 对象的类型转换

09 多重继承

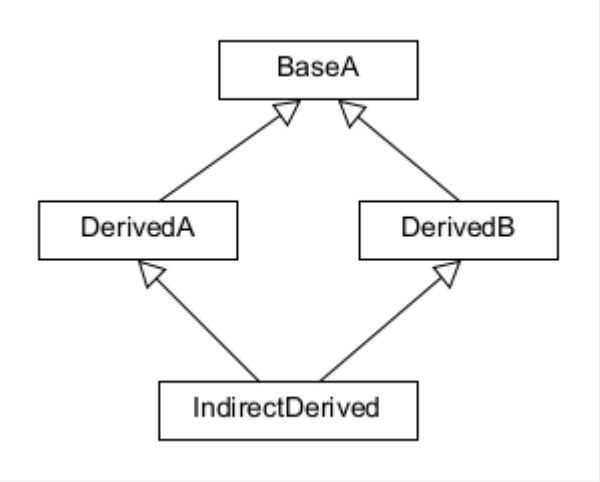
C++支持的多继承



多重继承：继承多个基类

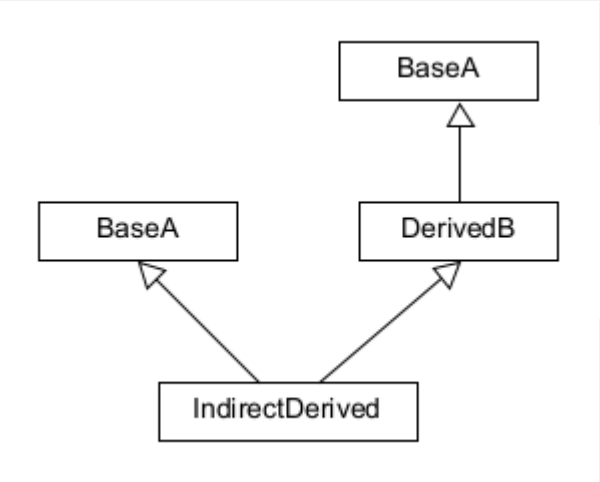


X



重复继承：菱形继承

多重继承特例，baseA被派生两次以上



X

多继承

- 多重继承：派生类有多个基类。
- 代表概念：C既是A又是B。

```
class 派生类名：继承访问控制1  基类名1，继承访问控制2  基类名2，...
{
        成员声明；
}
```

多继承例子——Device

```
class Device1 {
public:
        Device1() {
          volume = 5;
          powerOn = false;
        }
        Device1(int vol, bool onOrOff) {
          volume = vol;
          powerOn = onOrOff;
        }
        void showVol() {
          cout << "Volume is " << volume << endl;
        }

protected:
        int volume;                 // 音量
        bool powerOn;             // 开关状态
};
```

```
class Device2 {
public:
          Device2()
         {
          talkTime = 10;
          standbyTime = 300;
          power = 100;
         }

          Device2(int newTalkTime, int newStandbyTime, float powerCent)
         {

          talkTime = newTalkTime;
          standbyTime = newStandbyTime;
          power = powerCent;
         }
```

多继承例子——Device

```
class Device2 {
public:
     void showProperty()
    {
         cout << "The property of the device : "<< endl;
         cout << "talk time: " << talkTime << " hours" <<endl;
         cout << "standbyTime: " << standbyTime << " hours" <<endl;
     }

     void showPower ()
     {    cout <<" Power: " << power << endl;    }

protected:
       int  talkTime;                          //可通话时间（小时）
       int  standbyTime;                     //可待机时间（小时）
       float power;                            //剩余电量百分比
};
```

多继承例子——Device

```
class DeviceNew: public Device1, public Device2 {
public:
          DeviceNew()
         {     weight = 0.56;  }

          DeviceNew(float newWeight, int vol, bool onOrOff, int newTalkTime,
                                  int newStandbyTime, float powerCent) :
          Device2(newTalkTime, newStandbyTime, powerCent),
          Device1(vol, onOrOff)
         {    weight = newWeight;     }

          float getWeight()
         {     return weight;   }

private:
        float weight;                  // 重量（克）
};
```

多继承例子——Device

```
int main()
{
    DeviceNew  device(0.7, 3, false, 10, 250, 80);          //声明派生类对象

    // getWeight()函数是DEVICE_NEW类自身定义的
    cout << "The weight of the device : " <<device.getWeight()<<endl;

    // showVol()函数是从DEVICE1类继承来的
    device.showVol();

    // showProperty()函数是从DEVICE2类继承来的
    device.showProperty();

     return 0;
}
```

```
The weight of the device : 0.7
Volume is 3
The property of the device :
talk time: 10 hours
standbyTime: 250 hours
```

虚基类

继承基类时，在继承访问控制前添加保留字“virtual”。 那么这个基类就是一个虚拟基类。虚拟基类用于共享继承。

普通基类与虚基类之间的唯一区别只有在派生类重复继承了某一基类时才表现出来。

- 若派生类有一个虚基类作为祖先类，则在派生类构造函数中需要列出对虚基类构造函数的调用（否则，调用虚基类的默认构造函数），且对虚基类构造函数的调用总是先于普通基类的构造函数。
- 创建后代类对象时，当该后代类列出的虚基类构造函数被调用，Virtual关键字保证了虚基类的唯一副本只被初始化一次。
- 创建派生类对象时构造函数的调用次序：
  - 最先调用虚基类的构造函数；
  - 其次调用普通基类的构造函数，多个基类则按派生类声明时列出的次序、从左到右调用，而不是初始化列表中的次序；
  - 再次调用对象成员的构造函数，按类声明中对象成员出现的次序调用，而不是初始化列表中的次序
  - 最后执行派生类的构造函数。

虚基类

```
class Base {public:  int i;};
class Base1: virtual public Base {
public:int j;
};
class Base2: virtual public Base {
public:int k;
};
class Derived: public Base1, public Base2 {
public:int sum;
};
int main() {
    Derived obj;     // 声明一个派生类对象
    obj.i = 3;       // 正确：从Base继承的i在Derived中只有一份
    obj.j = 5;       // 正确：使用从Base1继承的j
    obj.k = 7;       // 正确：使用从Base2继承的k
    return 0;
}
```

虚基类

```
class baseA
{
public:
        baseA()
        {
                cout << endl << "This is baseA class." << endl;
        }
};

class baseB
{
public:
        baseB()
        {
                cout << endl << "This is baseB class." << endl;
        }
};
```

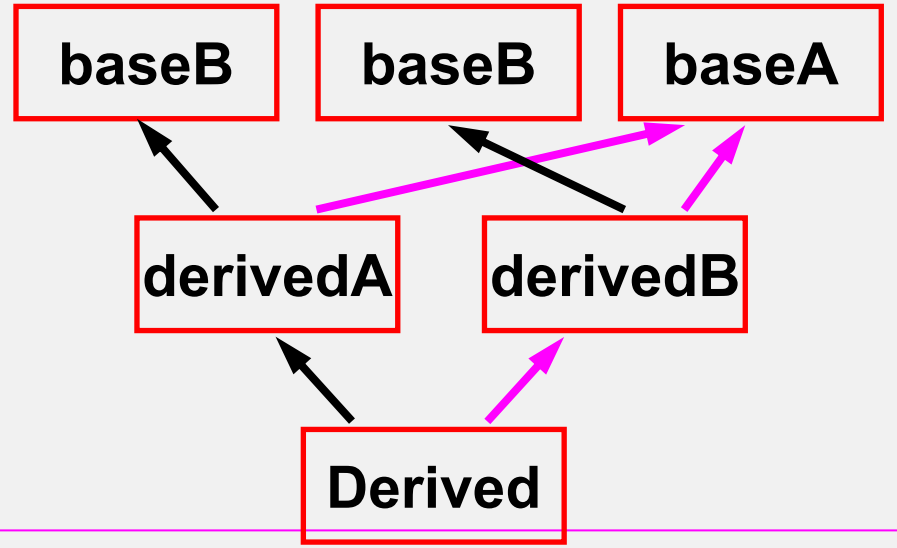
虚基类

```
class derivedA : public baseB, virtual public baseA
{
public:
        derivedA()
        {
                cout << endl << "This is derivedA class." << endl;
        }
};
class derivedB : public baseB, virtual public baseA
{
public:
        derivedB()
        {
                cout << endl << "This is derivedB class." << endl;
        }
};
```

虚基类

```
class Derived : public derivedA, virtual public derivedB
{
public:
        Derived()
        {
                cout << endl << "This is Derived class." << endl;
        }
};

int main()
{
        Derived obj;
        return 0;
}
```



虚基类

运行结果：

This is baseA class.

This is baseB class.

This is derivedB class.

This is baseB class.

This is derivedA class.

This is Derived class.

构造顺序：
1. 先基类后成员
2. 先虚后实
3. 先左后右

析构顺序：
与构造顺序相反

总结

- 基类与派生类
- 访问控制与恢复
- 继承中的构造与析构函数
- 继承对象的重载、屏蔽与兼容性问题
- cast
- 多继承
- 虚基类

中山大學
SUN YAT-SEN UNIVERSITY



谢谢

中山大学计算机学院

中山大学MOOC课程组